

COLEGIO DE CIENCIAS E INGENIERÍAS
INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN

Planificación para el Desarrollo del Proyecto Integrador

AgentProof: inducción automática de escenarios multi-turno desde trazas de interacción y reejecución adaptativa con usuarios simulados para la detección de regresiones en agentes basados en LLM

Autor: Jarod Styph Tierra Medina

Tutor: Ricardo Flores Moyano

Quito - Ecuador

2026

1. Título del Proyecto

AgentProof: inducción automática de escenarios multi-turno desde trazas de interacción y reejecución adaptativa con usuarios simulados para la detección de regresiones en agentes basados en LLM.

2. Relevancia y Justificación

Los agentes basados en modelos de lenguaje (LLM), es decir, sistemas que ejecutan tareas de extremo a extremo combinando un modelo con acceso a herramientas, constituyen el área de mayor crecimiento en la industria del software. Su adopción, sin embargo, enfrenta un problema abierto documentado en la literatura como la brecha de evaluación (evaluation gap): los agentes superan las evaluaciones internas de sus desarrolladores y luego fallan en producción, porque los métodos de evaluación existentes no reflejan de manera confiable el desempeño en condiciones reales [9], [10].

El ecosistema de herramientas ha madurado con rapidez y sería incorrecto afirmar que ninguna cubre el ciclo entre trazas, evaluación y regresión. LangSmith dispone de reglas de automatización que incorporan trazas, e incluso hilos conversacionales completos, a un conjunto de evaluación cuando cumplen condiciones definidas, como una etiqueta o una puntuación de retroalimentación baja [18]. La misma plataforma incorpora además simulación multi-turno: su biblioteca de evaluadores orquesta conversaciones entre el agente y un usuario simulado por un modelo, que adapta sus mensajes turno a turno, y califica la trayectoria completa; la configuración, la persona y el objetivo que guían a ese usuario simulado se proporcionan como entrada de evaluación, ya sea escritos directamente o tomados de un conjunto de datos, y no se inducen de forma automática a partir de la traza observada [24]. Braintrust permite construir conjuntos de datos de evaluación a partir de logs de producción y de trazas, ya sea añadiéndolas directamente desde su interfaz de registros y de revisión, donde el campo de entrada del span pasa a ser la entrada del conjunto de datos y su salida el valor esperado, ya sea solicitándoselo a su asistente en lenguaje natural [21]. Microsoft Foundry incorpora simuladores capaces de sostener conversaciones multi-turno con el agente bajo prueba, incluido un simulador adversarial para escenarios de seguridad, y ejecutar evaluaciones en la nube [17]. En el ámbito de código abierto, EvalView graba el comportamiento actual del agente como línea base dorada y detecta desviaciones mediante diferencias de instantáneas sobre las herramientas invocadas, su orden y su salida [16]. En el ámbito académico, el trabajo de testing estructural de agentes convierte trazas de OpenTelemetry en pruebas automatizadas con aserciones y emplea mocking del modelo para forzar un comportamiento reproducible [22].

La brecha no está en automatizar la captura ni la incorporación de casos desde trazas, que varias de estas herramientas ya resuelven, sino en la naturaleza del caso resultante y en la forma de reejecutarlo. En LangSmith y en Braintrust el artefacto que se genera automáticamente desde las trazas es un dato fijo, la traza o el hilo registrado, que se evalúa tal como quedó grabado;

la simulación multi-turno de LangSmith, en cambio, no se alimenta de esas trazas sino de una configuración de usuario provista como entrada de evaluación, de modo que ambas capacidades existen en la misma plataforma pero no están enlazadas entre sí. En EvalView la reejecución es estática sobre la entrada registrada, de modo que un camino distinto pero legítimo se reporta como desviación y una degradación que solo se manifiesta cuando el usuario reacciona de otra forma nunca llega a ejercitarse. En trace2test el artefacto inducido es un contrato sobre la trayectoria, es decir, secuencias de herramientas, argumentos, salidas y políticas, y su reejecución repite la entrada registrada, ya sea contra un mock o contra el agente real, sin que exista un interlocutor que reaccione a lo que la nueva versión responde; en consecuencia, un camino distinto pero igualmente válido puede violar el contrato, y una degradación que solo se manifiesta cuando el usuario replantea su petición no llega a ejercitarse. El trabajo de testing estructural privilegia de forma aún más explícita el determinismo, sustituyendo el modelo por un mock, lo que elimina por diseño la variabilidad conversacional que este proyecto busca precisamente ejercitar. Microsoft Foundry sí sostiene conversación multi-turno adaptativa, pero sus simuladores se dirigen por descripciones de escenario redactadas por el ingeniero, no por un objetivo inducido de las trazas del agente evaluado.

Contribución de AgentProof. Ninguno de los dos ingredientes por separado constituye una novedad. La construcción automática de casos de prueba a partir de trazas ya existe, por ejemplo en trace2test, que genera aserciones, y en Braintrust, que construye conjuntos de datos de evaluación desde los logs. La reejecución multi-turno adaptativa mediante usuarios simulados también existe, en Microsoft Foundry y en LangSmith, con la configuración del usuario provista como entrada de evaluación. Lo que no se identificó en la búsqueda realizada es la combinación de ambos: que el artefacto inducido desde la traza no sea un contrato sobre la trayectoria sino un escenario conversacional, con objetivo de usuario y criterios de éxito verificables, y que ese objetivo condicione a un usuario simulado encargado de generar una conversación nueva que se adapte a las respuestas de la versión evaluada, para después comparar versiones y detectar regresión con integración a CI. Esa combinación, y no cada pieza por separado, es la contribución que el proyecto reclama. El proyecto aporta, además, algo que las plataformas comerciales no publican: la validación empírica de esa metodología mediante experimentos con verdad de referencia conocida y métricas cuantitativas.

3. Objetivos

a. General

Diseñar, implementar y validar empíricamente un sistema (AgentProof) capaz de inducir automáticamente escenarios de prueba multi-turno a partir de trazas de interacción de un agente basado en LLM, reejecutarlos de forma adaptativa mediante usuarios simulados y detectar degradación de comportamiento entre versiones del agente, con un núcleo independiente del framework de orquestación empleado.

b. Específicos

- Implementar el pipeline completo y automático de extremo a extremo: captura de trazas, inducción de escenarios (objetivo del usuario, mensaje inicial y criterios verificables de éxito), reejecución adaptativa con usuarios simulados, evaluación LLM-as-a-Judge ciega y reporte de regresión con código de salida apto para CI/CD.
- Diseñar un esquema de trazas neutral y un protocolo de sesión que desacoplen el pipeline de los frameworks de agentes, e implementar adaptadores para cuatro frameworks que representan paradigmas de orquestación distintos (Claude Agent SDK, LangGraph, CrewAI y smolagents), cuantificando el costo de incorporación de cada uno y el alcance de las modificaciones que requiera el núcleo.
- Construir un corpus semilla de exactamente 20 conversaciones de interacción elicitada con el agente de caso de estudio, número fijado para mantener la coherencia con el protocolo estadístico pre-registrado, mediante un protocolo de participantes múltiples y una matriz de cobertura diseñada para que los comportamientos definidos queden todos presentes y para asegurar la viabilidad del diseño experimental.
- Formalizar seis degradaciones controladas como operadores de mutación [11], declarando para cada una el comportamiento que se espera afectar, el conjunto de escenarios expuestos mediante una regla computable sobre información fijada antes de la inducción, es decir, la tarjeta de situación y la traza semilla, y el predicado que verifica sobre las ejecuciones si la mutación produjo efectivamente el fallo esperado.
- Evaluar la capacidad de detección con un diseño estadístico pre-registrado: unidad de análisis, número de repeticiones justificado como compromiso entre sensibilidad y costo, criterio de regresión con control de tasa de falso descubrimiento [13], y cálculo de TPR y FPR sobre matrices de confusión a nivel de escenario, con control negativo de línea base contra sí misma.
- Validar la confiabilidad del evaluador automático frente a tres anotadores humanos mediante κ de Fleiss [12], matriz de κ de Cohen por pares y concordancia del evaluador con el voto mayoritario, y medir la fidelidad de los escenarios inducidos respecto a las conversaciones que los originaron.
- Replicar el experimento de detección en un segundo framework para evidenciar la portabilidad del sistema, manteniendo constantes dominio, herramientas, base de conocimiento y modelo interno.
- Publicar el sistema como software de código abierto con documentación, pruebas automatizadas, protocolo pre-registrado y resultados reproducibles.

4. Estado del Arte

Conceptos principales. Un agente LLM es un sistema donde un modelo de lenguaje interactúa con herramientas externas para completar tareas de múltiples pasos. La literatura de diseño

distingue entre workflows, cuyo flujo de control está predefinido en código, y agentes propiamente dichos, donde el modelo dirige su propio proceso en un bucle con retroalimentación del entorno [8]. Su evaluación se apoya en dos técnicas centrales: la simulación de usuarios, en la que un LLM interpreta al usuario para sostener conversaciones de prueba, y el patrón LLM-as-a-Judge, en el que un modelo califica ejecuciones contra criterios definidos y cuya confiabilidad debe a su vez validarse contra jueces humanos [7]. Para la caracterización de fallos se dispone de taxonomías específicas: la del AI Red Team de Microsoft cubre agentes individuales que usan herramientas [14], y trabajos recientes documentan fallos silenciosos en runtimes de agentes en producción [20].

Herramientas de evaluación y regresión. La siguiente tabla ubica a AgentProof frente a las soluciones existentes según el eslabón del ciclo que automatizan:

Herramienta	Origen del caso de prueba	Forma de reejecución	Regresión y CI
LangSmith [18], [24]	Dos vías: reglas de automatización que incorporan trazas o hilos, y configuración del usuario simulado provista como entrada de evaluación	Dos modos: evaluación del hilo registrado como dato fijo, y simulación multi-turno adaptativa con usuario simulado	Sí
Braintrust [21]	Conjuntos de datos de evaluación contruidos desde logs de producción y trazas, por interfaz o mediante su asistente	El caso es un par de entrada y salida esperada derivado del span registrado	Sí
Microsoft Foundry [17]	Descripciones de escenario y consultas provistas por el ingeniero	Simulación multi-turno adaptativa	Sí
EvalView [16]	Instantánea del comportamiento actual (línea base dorada)	Reejecución estática de la entrada grabada	Sí
trace2test [25]	Un modelo propone aserciones desde la traza: secuencias de herramientas, argumentos, salidas y políticas	Replay de la entrada registrada, contra mock determinista o contra el agente real; sin usuario que adapte la conversación	Sí
Testing estructural [22]	Trazas OpenTelemetry convertidas en pruebas con aserciones	Reejecución con el modelo sustituido por un mock, para forzar determinismo	Sí
agentevals	No genera casos; califica trazas existentes	No reejecuta	Parcial
AgentProof	Inducción automática desde trazas: objetivo, mensaje inicial y criterios verificables	Reejecución adaptativa con usuario simulado dirigido por objetivo	Sí

El proyecto de código abierto trace2test merece un examen aparte por ser el más próximo a la propuesta. Convierte trazas de ejecución de agentes en pruebas de regresión reproducibles: un modelo de lenguaje lee cada traza y propone aserciones, que quedan como especificaciones editables y legibles sobre secuencias de llamadas a herramientas, restricciones de argumentos, requisitos de salida e invariantes de política, del tipo no invocar el reembolso cuando la

verificación de identidad ha fallado. Su ejecución admite tanto un mock determinista y sin conexión como la invocación del agente real por punto HTTP o por comando, aplica primero las comprobaciones deterministas y reserva un evaluador basado en modelo para las comprobaciones semánticas, y produce reportes para integración continua en varios formatos. La generación automática del caso de prueba a partir de la traza, por tanto, tampoco constituye una novedad de este proyecto [25].

Trabajos académicos. τ -bench [1] y sus sucesores constituyen el estándar de evaluación de agentes con usuarios simulados y aportan la métrica pass^k para medir confiabilidad en múltiples intentos. τ^2 -bench [2] extendió el escenario a un entorno de control dual, en el que usuario y agente actúan sobre el mismo entorno, y la tercera generación, τ^3 -bench [26], amplía la cobertura con un dominio de conocimiento bancario que incorpora recuperación de información, correcciones sobre las tareas anteriores y evaluación de voz. La línea completa comparte, no obstante, un rasgo relevante para este proyecto: sus escenarios son conjuntos fijos escritos manualmente por investigadores, con estados finales anotados a mano, y no se derivan de las trazas del agente que se evalúa. ALMITA [3] es el trabajo más cercano en generación automática de pruebas para agentes de soporte, pero parte de procedimientos especificados por humanos y evalúa de forma estática por turnos; su resultado principal, que los agentes logran entre 75% y 99% de acierto en turnos aislados pero solo entre 1.6% y 15.6% en conversaciones completas, motiva directamente la reejecución conversacional adoptada en este proyecto. El Agent-Testing Agent [4] explora fallos mediante un meta-agente adversarial que analiza el código del agente objetivo; sus autores señalan explícitamente que no utiliza trazas reales ni realiza comparación de regresión entre versiones. En cuanto a las limitaciones del método, Lost in Simulation [5] documenta que los usuarios simulados son proxies imperfectos de usuarios humanos, con variaciones de hasta nueve puntos porcentuales según el modelo simulador, y Mind the Sim2Real Gap [6] caracteriza esa miscalibración; ambas críticas se incorporan al diseño fijando el modelo simulador como variable controlada y validando el evaluador contra anotadores humanos. MAST [15] valida un evaluador LLM contra anotadores humanos alcanzando $\kappa = 0.88$, precedente metodológico que este proyecto sigue en su experimento de concordancia; su taxonomía, en cambio, fue desarrollada para sistemas multi-agente y por ello no se emplea como anclaje de las degradaciones.

Brecha identificada. Dentro de las herramientas y trabajos identificados mediante el protocolo de búsqueda documentado, no se encontró ninguno que induzca desde la traza un escenario conversacional, con objetivo de usuario y criterios de éxito verificables, y que ese objetivo condicione a un usuario simulado encargado de generar una conversación nueva que se adapte a las respuestas de la versión evaluada. La formulación se limita de forma deliberada a lo que la búsqueda permite sostener y no afirma inexistencia universal. Las soluciones identificadas automatizan eslabones distintos: la incorporación automática de trazas o hilos al conjunto de evaluación (LangSmith), la construcción de conjuntos de datos de evaluación desde logs de producción y trazas (Braintrust), la generación automática de aserciones sobre la trayectoria a partir de la traza (trace2test), la captura y comparación por instantáneas (EvalView), la simulación multi-turno adaptativa cuya configuración se provee como entrada de evaluación (Foundry y

LangSmith) y la conversión de trazas en pruebas con el modelo sustituido por un mock (testing estructural). La contribución de este proyecto se ubica en la combinación descrita y en su validación empírica.

5. Metodología de Trabajo

El proyecto se desarrolla en seis fases: revisión del estado del arte, de la cual ya se completó una primera iteración con protocolo de búsqueda documentado y que se replicará en bases académicas con acceso institucional; diseño del sistema; implementación iterativa con desarrollo guiado por pruebas; construcción del corpus semilla; pre-registro del protocolo experimental; y validación experimental con redacción del documento final. Se aplican los principios del diseño de ingeniería (separación de responsabilidades mediante interfaces explícitas, desarrollo guiado por pruebas, control de versiones con integración continua y documentación pública), y el esquema de trazas se mantiene conceptualmente alineado con las convenciones semánticas GenAI de OpenTelemetry [19], documentando la correspondencia campo a campo como ruta de adopción futura.

5.1 Patrón de diseño del sistema

Siguiendo la taxonomía de referencia [8], AgentProof se define como un workflow híbrido: un pipeline determinista de encadenamiento cuyo control de flujo reside completamente en código, con cognición LLM delegada en tres puntos. Dos son transformaciones de una sola pasada con salida estructurada: el inductor de escenarios y el evaluador ciego, que nunca conoce la versión del agente que califica. El tercero es la única etapa genuinamente agéntica: el usuario simulado, un agente conversacional de autonomía acotada, con un bucle de percepción, decisión y acción, un presupuesto máximo de turnos y una condición de parada explícita. La agencia se introduce ahí, y solo ahí, por necesidad: el agente evaluado es no determinista, de modo que una reejecución guionada se rompería; la ruta de la conversación no puede predefinirse y el usuario simulado debe decidir cada turno en función de la respuesta recibida. El resto del pipeline permanece determinista bajo el principio de que el instrumento de medición debe ser más predecible que el objeto medido.

El sistema incorpora supervisión humana en cuatro puntos de control: la elicitación del corpus semilla, la revisión opcional de escenarios, la validación del evaluador con anotadores y el gate de CI, que bloquea e informa pero nunca decide el despliegue ni modifica al agente. No se emplea autocrítica iterativa (reflection) en la versión inicial del inductor ni del evaluador, por costo, estabilidad y medibilidad; se propone como extensión una ablación que compare evaluador de una pasada frente a evaluador con autocrítica, midiendo el cambio en la concordancia con anotadores humanos.

5.2 Corpus semilla: origen y protocolo de elicitación

El corpus de partida está compuesto por exactamente 20 conversaciones de interacción genuina con el agente desplegado, no por conversaciones sintetizadas a partir de especificaciones. El número se fija de antemano y no se expresa como mínimo, porque el diseño estadístico, la corrección por comparaciones múltiples, el volumen experimental y el experimento de fidelidad

están todos formulados sobre veinte escenarios; alterarlo obligaría a recalcular el protocolo pre-registrado. La correspondencia es de uno a uno: de cada conversación semilla se induce exactamente un escenario, de modo que las veinte conversaciones producen veinte escenarios, y esos veinte escenarios son las veinte hipótesis que se contrastan por operador de mutación, lo que fija en veinte el número de comparaciones sobre el que se aplica la corrección de Benjamini-Hochberg. Si alguna conversación no permitiera inducir un escenario utilizable, se elicitaba una conversación de reemplazo hasta completar las veinte, en lugar de reducir el tamaño del conjunto. Su elicitación sigue un protocolo explícito: participan entre cuatro y seis personas (el autor y estudiantes de la carrera ajenos al proyecto), cada una de las cuales recibe únicamente una tarjeta de situación (por ejemplo, "acabas de recibir un producto defectuoso y quieres tu dinero de vuelta") sin instrucciones sobre cómo conversar, y dialoga libremente con el agente. Las sesiones se distribuyen según una matriz de cobertura que cruza tres dimensiones: intención del usuario, tono o estilo conversacional (directo, confundido, insistente) y resultado esperado (procede, no procede, requiere escalamiento). La matriz se diseña de modo que cada operador de degradación de la Sección 5.4 disponga de un subconjunto no trivial de escenarios expuestos y de otro de escenarios no expuestos que sirva de control, requisito que vincula el diseño del corpus con el diseño estadístico. Los atributos de cada tarjeta, es decir intención, tono y resultado esperado, se registran junto con la sesión y constituyen información fijada antes de la inducción, lo que permite emplearlos después como verdad de referencia independiente del sistema evaluado, según se detalla en la Sección 5.4.

Alcance de la afirmación. Estas conversaciones representan interacción humana genuina con el agente, pero no provienen de un sistema en producción con usuarios finales. En consecuencia, el documento emplea el término "trazas de interacción elicitada" y no "trazas de producción". La generalización hacia tráfico de producción se declara como limitación del estudio y como trabajo futuro.

5.3 Grado de automatización

El pipeline opera de forma completamente automática de extremo a extremo, y esa es su modalidad de funcionamiento normal: no requiere intervención humana entre la captura de trazas y el reporte de regresión. La revisión humana de escenarios no forma parte de ese funcionamiento normal; se emplea como instrumento experimental para medir la fidelidad de la inducción y, adicionalmente, se expone como una función opcional de la herramienta para quien desee ejercerla. En consecuencia, las métricas primarias de detección se calculan sobre la suite generada automáticamente y sin curar, de modo que midan el sistema automático y no un híbrido humano-máquina; la suite curada se emplea únicamente como análisis secundario, para cuantificar cuánto mejoraría el sistema bajo supervisión opcional.

5.4 Degradaciones controladas como operadores de mutación

Las degradaciones se formulan como operadores de mutación en el sentido del testing de mutación clásico [11]: modificaciones controladas del sistema bajo prueba cuyo propósito es medir la

capacidad de detección de una suite. Esta analogía aporta el vocabulario preciso para una dificultad inherente: una mutación no necesariamente produce fallo en todo escenario, y aquellos escenarios en los que la mutación no puede manifestarse son análogos al concepto de mutante equivalente y no deben computarse como falsos negativos. Cada operador declara su mecanismo, el comportamiento que se espera afectar y el conjunto expuesto, definido mediante una regla computable sobre información fijada antes de la inducción del escenario, y el conjunto manifestado, que se deriva del anterior mediante un predicado de fallo verificado sobre las ejecuciones, ambos detallados más adelante en esta misma sección. La semántica de los fallos se apoya en la taxonomía de modos de fallo en sistemas agénticos [14], [20].

Op.	Mecanismo y comportamiento afectado	Conjunto expuesto E_exp (fijado antes de la inducción)	Familia
D1	Se elimina la herramienta de consulta de pedidos: el agente no puede obtener datos reales y el fallo típico es inventarlos	Escenarios cuya traza semilla contiene al menos una llamada a la consulta de pedidos	A
D4	La herramienta de política de reembolsos devuelve datos corruptos: el agente propaga el error sin verificarlo	Escenarios cuya traza semilla contiene al menos una llamada a la política de reembolsos	A
D6	El límite de pasos se reduce a dos: truncamiento antes de completar tareas de varios pasos	Escenarios cuya traza semilla contiene tres o más llamadas a herramientas	A
D2	Se recorta el prompt de sistema: disminuye la propensión a verificar la política antes de decidir	Escenarios cuya tarjeta de situación plantea una decisión de reembolso	B
D5	Se inyecta una instrucción contradictoria (aprobar todo reembolso): invierte la decisión donde no procede	Escenarios cuya tarjeta de situación declara como resultado esperado que el reembolso no procede	B
D3	Se degrada el modelo del agente: menor capacidad de razonamiento multi-paso, con efecto difuso	No se define conjunto expuesto; se analiza como efecto de tamaño sobre la suite completa	B

Independencia del conjunto expuesto respecto del sistema evaluado. La pertenencia de un escenario al conjunto expuesto se determina exclusivamente a partir de información fijada antes de que AgentProof induzca el escenario: la tarjeta de situación entregada al participante, cuyos atributos de intención y resultado esperado se registran al diseñar la matriz de cobertura, y la traza semilla, que registra qué herramientas se invocaron y cuántos pasos requirió la conversación. Ninguna regla depende de los metadatos que el sistema produce al inducir el escenario. Esta separación es deliberada: si la pertenencia al conjunto se derivara del escenario inducido, el propio sistema estaría determinando en parte la verdad de referencia con la que se lo evalúa, y las métricas de detección perderían validez.

Exposición no equivale a regresión: el conjunto manifestado. Pertenecer al conjunto expuesto significa que la mutación pudo afectar al escenario, no que necesariamente lo haya hecho. Un agente al que se le reduce el límite de pasos puede encontrar una trayectoria alternativa más corta y válida; uno al que se le recorta el prompt puede conservar la conducta correcta por otras vías. Tratar toda no detección sobre el conjunto expuesto como falso negativo penalizaría al sistema por escenarios en los que no había nada que detectar, que es el análogo del mutante equivalente del testing de mutación [11]. Se define por ello un segundo conjunto, el conjunto manifestado, que es el subconjunto del expuesto en el que se verifica que la mutación produjo el fallo que pretende inducir.

Qué cuenta como manifestación: resultado incorrecto, no trayectoria distinta. La manifestación no se declara por un cambio de trayectoria. Que una herramienta deje de invocarse, que varíe el número de pasos o que el agente se apoye en información distinta son cambios de comportamiento que pueden dejar intacto el resultado para el usuario, y en ese caso no hay regresión que detectar. La condición exigida es que el desenlace comunicado al usuario sea incorrecto respecto de una verdad fijada con independencia del sistema. Esa verdad tiene tres fuentes, todas anteriores a la ejecución y ajenas a AgentProof: la base de conocimiento controlada del agente de caso de estudio, que fija el estado real de cada pedido y el texto verdadero de la política de reembolsos; los atributos declarados en la tarjeta de situación, entre ellos el resultado esperado; y los eventos de la propia traza, como las llamadas a herramientas y el motivo de terminación. En ningún caso interviene el evaluador automático de AgentProof ni los criterios que el sistema induce, precisamente porque su capacidad de acierto es lo que el estudio mide.

Predicados de fallo por operador. D1: la respuesta final enuncia un atributo del pedido, es decir estado, fecha o importe, cuyo valor discrepa del registrado en la base de conocimiento, o bien omite el atributo que la tarjeta pedía obtener. D2, D4 y D5 comparten predicado, porque los tres pretenden inducir una decisión equivocada aunque por mecanismos distintos: la decisión sobre el reembolso comunicada al usuario contradice la que se deriva de aplicar la política verdadera de la base de conocimiento al estado real del pedido, decisión que la tarjeta de situación ya declara de antemano. D6: la conversación termina por agotamiento del límite de pasos y, además, el desenlace que la tarjeta pedía no aparece en la respuesta final, lo que excluye de forma expresa los casos en que el agente alcanzó una trayectoria más corta y válida. D3 no define conjunto expuesto y por tanto tampoco admite predicado de manifestación.

Extracción del desenlace y límite de la automatización. Los predicados anteriores requieren extraer de la respuesta final dos elementos: la decisión comunicada, dentro de un vocabulario cerrado de tres valores que son procede, no procede y escalamiento, y los valores de atributos de pedido, que se contrastan contra la base de conocimiento. La extracción se realiza mediante reglas sobre ese vocabulario cerrado y sobre los valores conocidos de la base. Se declara de forma explícita el límite de este procedimiento: no toda respuesta en lenguaje natural resuelve por regla, y los casos que la extracción no resuelva se anotan manualmente por el autor contra la base de conocimiento y la tarjeta, se registran como tales y se reportan en el recuento. Esa anotación se

realiza sobre respuestas anonimizadas respecto de su procedencia, es decir, sin indicar si corresponden a la línea base o a una versión mutada, y presentadas en orden aleatorio, para reducir el sesgo del anotador. Esa anotación manual es humana y determinista frente a una verdad fijada, y en particular no emplea el evaluador automático cuya validez se está midiendo.

La manifestación es diferencial: se compara contra la línea base. El predicado de fallo se evalúa de forma independiente en cada una de las cinco ejecuciones candidatas del escenario y, con idéntico procedimiento, en las cinco ejecuciones de la línea base de referencia, lo que produce dos tasas de fallo por escenario con valores múltiplos de dos décimas. Evaluar el predicado únicamente sobre el candidato sería insuficiente, porque un escenario en el que el agente ya fallaba antes de la mutación no representa una regresión atribuible a ella. Esta evaluación adicional no incrementa el volumen experimental, ya que se aplica sobre ejecuciones de línea base que el diseño ya contempla; para evitar ambigüedad se fija que la primera de las dos corridas de línea base actúa como referencia en todas las comparaciones con versiones candidatas, y la segunda queda reservada al control negativo de la tasa de falsos positivos.

Regla de agregación pre-registrada. Un escenario expuesto pertenece al conjunto de regresión manifestada cuando la tasa de fallo del candidato supera a la de la línea base de referencia en al menos cuatro décimas. Esa es la única condición exigida. La regla se fija en el pre-registro antes de ejecutar cualquier experimento y no se ajusta después de observar los resultados.

Por qué una sola condición, y por qué ese umbral. La verdad de referencia se establece con independencia de lo que el detector alcance a marcar. Añadir una condición que restringiera el conjunto a los escenarios que el criterio estadístico está en condiciones de declarar volvería la tasa de detección una medida casi circular, calculada solo sobre los casos que el propio método puede resolver, y la elevaría de forma artificial. La diferencia de cuatro décimas se adopta porque es el mismo tamaño mínimo de efecto que emplea el criterio de regresión, de modo que la referencia queda expresada en la misma escala de magnitud, pero se aplica sobre el predicado objetivo de fallo y no sobre los veredictos del evaluador, con lo que ambos planos permanecen separados. La diferencia respecto de la línea base sigue siendo imprescindible, porque un escenario que falla con la misma frecuencia antes y después de la mutación no constituye una regresión atribuible a ella.

Consecuencia asumida sobre la resolución del diseño. Un escenario con regresión objetiva que la prueba exacta de Fisher con corrección de Benjamini-Hochberg no consiga marcar se contabiliza como falso negativo real, y no se excluye del denominador. Refleja el límite de resolución que impone trabajar con cinco repeticiones, límite que conviene declarar con precisión: con cinco repeticiones por brazo ninguna comparación aislada alcanza el primer umbral de la corrección, situado en 0.0025 para veinte comparaciones, puesto que incluso el caso extremo de cero fallos en la línea base frente a cinco en el candidato produce un valor p de 0.0079. La declaración de regresión depende por tanto de que varios escenarios regresen de forma conjunta, como ya se detalla en el criterio de regresión, y la tasa de detección resultante debe leerse a la luz de esa restricción y no como una medida de la calidad intrínseca del evaluador.

Transparencia y análisis de sensibilidad. Se reporta la distribución conjunta de tasas de fallo de línea base y candidato por escenario y operador, que permite reconstruir cualquier otra clasificación a partir de los datos crudos, y la distribución descriptiva de frecuencias de fallo del candidato, de cero a cinco quintos. El análisis de sensibilidad, también pre-declarado, recalcula la tasa de detección con umbrales de diferencia de dos, cuatro y seis décimas, de modo que quede visible cuánto depende el resultado del valor elegido.

La familia A agrupa las mutaciones de mecanismo, en las que una capacidad se elimina o corrompe y el conjunto expuesto es por tanto inequívoco; sobre ella se calculan las métricas principales de detección. La familia B agrupa las mutaciones conductuales, que alteran la propensión del agente sin suprimir la capacidad: se reportan por separado, con su conjunto expuesto declarado cuando aplica y acompañadas del tamaño de efecto. El operador D3 se excluye del cómputo de TPR precisamente porque un cambio de modelo no produce fallo en todo escenario, y se analiza únicamente como efecto de tamaño. La verificación de manifestación descrita más arriba se aplica a ambas familias, dado que ni siquiera las mutaciones de mecanismo garantizan por sí solas una regresión observable en todos los escenarios expuestos.

5.5 Diseño estadístico

El protocolo se pre-registra antes de ejecutar cualquier experimento. La unidad de análisis es el escenario, no la ejecución individual: cada par (escenario, versión) produce una tasa de aprobación sobre R repeticiones independientes.

Número de repeticiones. Se fija $R = 5$ como compromiso entre sensibilidad y costo, y no como resultado de un análisis formal de potencia. El criterio que orienta la elección es la resolución que el número de repeticiones impone sobre la prueba exacta de Fisher en el caso extremo de inversión completa del comportamiento, es decir, todas las repeticiones aprobadas frente a ninguna. Con $R = 3$ ese caso alcanza $p = 0.100$ y con $R = 4$, $p = 0.029$, valores insuficientes frente a la corrección de Benjamini-Hochberg sobre veinte escenarios [13]. Con $R = 5$ alcanza $p = 0.0079$, que supera el umbral corregido cuando al menos cuatro escenarios regresan de forma conjunta, situación esperada ante una degradación real. Con $R = 6$ alcanza $p = 0.0022$ y supera incluso el umbral más estricto de la corrección, lo que permitiría declarar regresión en un escenario aislado. Se adopta $R = 5$ porque las degradaciones sistemáticas, que son el objeto del estudio, se manifiestan en conjuntos de escenarios y no en casos aislados, y porque pasar a $R = 6$ incrementa en un veinte por ciento el volumen de ejecuciones y su costo. La limitación se declara de forma explícita: con $R = 5$ una regresión que afecte a un solo escenario se reporta como señal débil y no como detección confirmada.

Criterio de regresión. Un escenario se declara regresado si se cumplen dos condiciones: la prueba exacta de Fisher entre línea base y versión candidata resulta significativa tras la corrección de Benjamini-Hochberg con $\alpha = 0.05$ sobre los veinte escenarios, y la caída absoluta de la tasa de

aprobación es de al menos 0.4. La segunda condición evita marcar como regresión diferencias triviales que resulten estadísticamente significativas.

Matriz de confusión. La clase verdadera de cada escenario la fija el predicado objetivo de fallo, que es evaluable sobre los veinte escenarios con independencia de que estuvieran expuestos o no, de modo que todos ellos reciben etiqueta y entran en la matriz. Es positivo el escenario con regresión manifestada, y es negativo todo aquel en el que no se verificó regresión objetiva, tanto si la mutación no podía afectarlo, por no estar expuesto, como si podía afectarlo y no llegó a degradar el desenlace. Un verdadero positivo es un escenario positivo marcado como regresión por el sistema, y un falso negativo, un escenario positivo no marcado; la tasa de detección es el cociente de verdaderos positivos sobre el total de positivos. Un falso positivo es un escenario negativo marcado como regresión, y un verdadero negativo, un escenario negativo no marcado; la tasa de falsos positivos es el cociente de falsos positivos sobre el total de negativos. Las cuatro celdas particionan los veinte escenarios sin solapamiento ni exclusiones.

Por qué los expuestos sin regresión cuentan como negativos. Si no se produjo regresión objetiva y el sistema marca una, se trata de un falso positivo, con independencia de que la mutación estuviera presente en ese escenario. Excluir del cálculo a los expuestos sin regresión manifestada dejaría fuera precisamente los negativos más exigentes, que son aquellos en los que la mutación sí alteró el comportamiento pero no degradó el desenlace, y en los que un detector que reaccionara a cualquier cambio de trayectoria, en lugar de a la degradación, se equivocaría. Reservar el cálculo a los escenarios no expuestos produciría por tanto una tasa de falsos positivos artificialmente favorable. Como análisis secundario se desglosa el recuento de falsos positivos entre los dos subgrupos de negativos, los no expuestos y los expuestos sin regresión manifestada, lo que permite distinguir el error frente a un caso inerte del error frente a un caso que cambió sin empeorar.

Control negativo. La medida más relevante de FPR proviene de comparar dos ejecuciones independientes de la misma versión línea base, con cinco repeticiones cada una y aplicando idéntico criterio: todo escenario marcado en esa comparación es un falso positivo atribuible a la variabilidad estocástica del sistema.

Volumen experimental. Ocho condiciones (dos líneas base independientes y seis mutantes) por veinte escenarios y cinco repeticiones producen 800 conversaciones en el framework principal, a las que se suman 400 de la replicación en el segundo framework (línea base y tres mutantes de familia A), totalizando aproximadamente 1200 conversaciones evaluadas y 1200 juicios automáticos.

Concordancia entre anotadores. La validación del evaluador emplea una muestra de 40 pares (escenario, traza) balanceada en veinte veredictos aprobados y veinte reprobados, con el fin de mitigar la paradoja de κ por prevalencia. La selección se realiza por muestreo aleatorio estratificado sobre el conjunto de ejecuciones ya evaluadas: se separan los pares según el veredicto automático y, dentro de cada estrato, se muestrea de forma que ningún escenario ni ninguna versión del agente aporte más de dos pares, evitando que un caso particular domine la medida. La semilla

del generador aleatorio se fija y se registra en el pre-registro, de modo que la muestra sea reproducible. El balance de veinte y veinte es una construcción destinada a estimar la concordancia con una prevalencia equilibrada y no representa la proporción natural de aprobaciones y reprobaciones del sistema, que se reporta por separado. Se prevé la contingencia de que alguno de los dos estratos no reúna veinte pares disponibles una vez aplicado el límite de dos por escenario y por versión: en ese caso se eleva ese límite a tres dentro del estrato deficitario y, si aun así no se alcanza el número, se reduce el tamaño de ambos estratos al del menor, conservando el balance y reportando el tamaño final junto con su efecto sobre la precisión de la estimación. En ningún caso se desbalancea la muestra para completar el total. Los anotadores reciben las transcripciones sin el veredicto automático y sin la etiqueta de versión del agente, de modo que califican a ciegas respecto de ambos; el mismo cegamiento se aplica en la calificación de fidelidad del experimento E3. Participan tres anotadores humanos independientes con una rúbrica idéntica a la del evaluador automático. Dado que se trata de más de dos calificadores, no corresponde un único κ de Cohen: se reporta κ de Fleiss [12] entre los tres anotadores como referencia de concordancia entre anotadores humanos, una matriz de κ de Cohen por pares entre anotadores como detalle, y κ de Cohen entre el evaluador automático y el voto mayoritario humano, que sí constituye una comparación entre dos calificadores. Cada coeficiente se acompaña del acuerdo bruto porcentual.

5.6 Selección y fijación de modelos

La asignación de modelos sigue tres reglas: calidad donde se valida la ciencia, economía donde hay volumen, y evaluador distinto y más capaz que el agente evaluado, para mitigar el sesgo de auto-preferencia documentado en la literatura de LLM-as-a-Judge [7]. Los identificadores se fijan de forma explícita, se centralizan en un único módulo del código y se registran junto a cada corrida, de modo que los experimentos sean reproducibles y auditables.

Componente	Identificador fijado	Justificación
Inductor de escenarios	claude-opus-5	La calidad de los criterios determina la validez de toda la suite; baja frecuencia de invocación
Evaluador (judge)	claude-opus-5	Instrumento de medición validado contra anotadores; distinto y más capaz que el agente evaluado
Usuario simulado	claude-haiku-4-5-20251001	Componente de mayor volumen; se mantiene fijo en todas las condiciones como variable controlada [5]
Agente objetivo (línea base)	claude-sonnet-5	Representa el despliegue típico de producción
Agente objetivo degradado (D3)	claude-haiku-4-5-20251001	Operador de degradación por reducción de capacidad

Los identificadores siguen la nomenclatura oficial de la API. Desde la generación 4.6, los modelos emplean identificadores sin fecha que constituyen por sí mismos una instantánea fijada; los modelos anteriores, como Claude Haiku 4.5, tienen un identificador con fecha y un alias sin ella. Se utiliza la forma con fecha cuando existe, por ser la que garantiza la reproducibilidad. Se deja

constancia además de que el compromiso de disponibilidad de Claude Haiku 4.5 se extiende al menos hasta octubre de 2026: si el modelo fuera retirado durante la ejecución experimental, se sustituiría por el modelo vigente de capacidad equivalente y se reejecutarían las condiciones afectadas, documentando el cambio en los resultados.

5.7 Portabilidad entre frameworks

El núcleo del sistema depende únicamente de un esquema de trazas neutral y de un protocolo de sesión; el conocimiento de cada framework se confina a un adaptador. Se implementan cuatro adaptadores que cubren paradigmas de orquestación distintos: Claude Agent SDK (SDK nativo), LangGraph (grafos), CrewAI (roles) y smolagents (tool-calling ligero). Los dos primeros se someten a evaluación experimental completa; los dos restantes se verifican mediante una ejecución del pipeline de extremo a extremo, sin repetición estadística, como evidencia de versatilidad del diseño.

Alcance de la afirmación. El objetivo de portabilidad no equivale a agnosticismo universal, y no se presenta como tal. Los cuatro adaptadores se encuentran implementados en el prototipo actual, lo que permite reportar dos observaciones preliminares medidas sobre él, cuyo detalle se presentará formalmente en el capítulo de resultados. La primera es que la incorporación del primer framework distinto del inicial exigió un refactor único del núcleo, que introdujo el protocolo de sesión y modificó las firmas del simulador y del ejecutor. La segunda es que, una vez realizado ese refactor, los dos frameworks siguientes requirieron 131 y 143 líneas respectivamente y no obligaron a modificar el simulador, el ejecutor, el evaluador ni el módulo de reporte; el único punto de cambio fue el despachador de la interfaz de línea de comandos. La lectura que se propone, y que el experimento de replicación deberá confirmar, es que el costo de independizar el sistema del framework se paga una sola vez y que el costo marginal de cada framework adicional es acotado. La verificación se realizó sobre el historial de control de versiones del repositorio: se identificaron los módulos modificados por cada confirmación a partir de la lista de archivos de sus diferencias, se contaron las líneas de cada adaptador sobre el árbol de trabajo, y se comprobó por inspección de importaciones que ningún módulo del núcleo declare dependencias de un framework de agentes. El término "agnóstico" se emplea como propiedad de diseño perseguida, no como resultado demostrado.

5.8 Experimentos

- **E1. Detección de degradaciones.** Aplicación de los seis operadores de mutación al agente de caso de estudio y medición de TPR y FPR según el diseño de la Sección 5.5, con control negativo de línea base contra sí misma y análisis de estabilidad entre repeticiones. Incluye $\text{pass}^k [1]$ como métrica de confiabilidad.
- **E2. Concordancia del evaluador.** Tres anotadores humanos sobre 40 pares balanceados; κ de Fleiss, matriz de κ de Cohen por pares y concordancia del evaluador con el voto mayoritario.

- **E3. Fidelidad de la inducción.** Los tres anotadores califican cada uno de los veinte escenarios inducidos, de modo que todos los escenarios reciben tres calificaciones independientes. La escala Likert de cinco puntos mide la fidelidad del escenario respecto de la conversación semilla que lo originó, con anclajes definidos: uno, el escenario no corresponde a la conversación; tres, corresponde parcialmente, con omisiones o criterios no verificables; cinco, recoge fielmente el objetivo del usuario y criterios verificables a partir de la conversación. El resultado se resume mediante la mediana por escenario y el porcentaje de escenarios con mediana igual o superior a cuatro, que constituye la métrica principal. Al tratarse de datos ordinales con tres calificadores, la concordancia se estima mediante el alfa de Krippendorff con ponderación ordinal [23], acompañado de las comparaciones por pares; no se emplea el kappa de Fleiss, apropiado para escalas nominales como la del experimento E2. Se declaran dos limitaciones del coeficiente en este diseño: con veinte ítems su intervalo de confianza es ancho, y si las calificaciones se concentran en los valores altos la baja varianza lo deprime, por lo que se reporta junto con la distribución completa de calificaciones. Los anotadores realizan E3 en una sesión separada de la de E2 para reducir el anclaje entre tareas. Se reporta además el porcentaje de escenarios que requerirían corrección bajo supervisión opcional.
- **E4. Replicación entre frameworks.** Repetición de E1 en el segundo framework manteniendo constantes dominio, herramientas, base de conocimiento y modelo interno, de modo que la única variable sea el framework de orquestación.

En todos los experimentos se reportan costo económico y latencia por escenario evaluado, desglosados por componente.

6. Sumario de Contenidos

- Introducción: contexto, problema, objetivos y contribuciones.
- Estado del arte: agentes LLM, evaluación de agentes, simulación de usuarios, LLM-as-a-Judge, taxonomías de fallos, herramientas existentes y brecha identificada.
- Descripción de la propuesta: arquitectura del pipeline, patrón de diseño híbrido, esquema de trazas, protocolo de sesión y adaptadores, selección de modelos.
- Desarrollo del prototipo: implementación de los módulos, agentes de caso de estudio, operadores de mutación y pruebas automatizadas.
- Diseño experimental y pre-registro: corpus semilla, conjuntos expuesto y manifestado, predicados de fallo, criterio de regresión y tamaño muestral.
- Experimentos y análisis de resultados: E1 a E4, con costos y latencias.
- Conclusiones y trabajo futuro: limitaciones declaradas, adaptadores adicionales, telemetría OpenTelemetry, ablación con autocritica y validación sobre tráfico de producción.

7. Recursos

a. Humanos

Estudiante (autor) y tutor del proyecto. Adicionalmente, entre cuatro y seis participantes voluntarios para la elicitación del corpus semilla (sesiones de aproximadamente diez minutos) y tres anotadores voluntarios que intervienen en dos tareas, la validación del evaluador automático y la calificación de fidelidad de los escenarios inducidos, realizadas en sesiones separadas para reducir el anclaje entre ambas, con una dedicación estimada de dos a tres horas por anotador en total y en modalidad asíncrona.

b. Materiales

Computador personal del estudiante. No se requiere hardware de laboratorio: toda la inferencia de modelos se realiza mediante servicios de API en la nube.

c. Económicos

El consumo de API se estima a partir del volumen experimental de la Sección 5.5: aproximadamente 1200 conversaciones evaluadas y 1200 juicios automáticos, más el consumo asociado al desarrollo y la depuración. Con un costo unitario estimado entre USD 0.04 y USD 0.09 por conversación evaluada según el modelo empleado, el presupuesto se estima entre USD 150 y USD 300, financiado por el estudiante; el costo real es a su vez una métrica reportada del sistema. Como palanca de contingencia se documenta la reducción de las repeticiones a tres en la replicación entre frameworks, que disminuye el volumen total en torno al 13% sin afectar el experimento principal. Antes de comprometer el volumen completo se ejecutará una corrida piloto reducida, de cinco escenarios, dos repeticiones y dos condiciones, con el fin de medir el costo y la latencia reales por conversación y ajustar la planificación si las cifras se desviaran de lo estimado. El software empleado no requiere licencias.

8. Cronograma de Actividades

El desarrollo se realiza en los cuatro meses del período lectivo, de septiembre a diciembre de 2026.

Actividades	Sep	Oct	Nov	Dic
A1: Documento de planificación y sus revisiones	X			
A2: Estado del arte, protocolo de búsqueda y réplica en bases académicas	X	X		
A3: Implementación del pipeline y de los adaptadores de framework	X	X		
A4: Corpus semilla elicitado y verificación del pipeline en vivo		X		
A5: Pre-registro del protocolo experimental y corrida piloto		X		
A6: E1 y E4, detección de degradaciones y replicación entre frameworks			X	

Actividades	Sep	Oct	Nov	Dic
A7: E2 y E3, concordancia del evaluador y fidelidad de la inducción			X	X
A8: Análisis estadístico de resultados				X
A9: Redacción del documento final			X	X
A10: Preparación de la defensa y entrega				X

9. Entregables

- Documento de planificación del proyecto integrador (este documento).
- Repositorio de código abierto de AgentProof: pipeline completo, adaptadores para cuatro frameworks, agentes de caso de estudio con sus operadores de mutación y suite de pruebas automatizadas.
- Corpus semilla de conversaciones elicítadas, con su matriz de cobertura, y suite de escenarios inducidos automáticamente.
- Protocolo experimental pre-registrado: criterio de regresión, conjuntos expuesto y manifestado por operador con sus predicados de fallo y su regla de agregación, y tamaño muestral.
- Resultados experimentales de E1 a E4: datos crudos, análisis estadístico y gráficas reproducibles.
- Documento final del proyecto integrador con los capítulos del sumario de contenidos.
- Presentación para la defensa del proyecto.

10. Referencias

- [1] S. Yao, N. Shinn, P. Razavi y K. Narasimhan, "τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains", arXiv:2406.12045, 2024.
- [2] V. Barres et al., "τ²-Bench: Evaluating Conversational Agents in a Dual-Control Environment", arXiv:2506.07982, 2025.
- [3] S. Arcadinho, D. Aparicio y M. Almeida, "Automated test generation to evaluate tool-augmented LLMs as conversational AI agents", 2nd GenBench Workshop (EMNLP), arXiv:2409.15934, 2024.
- [4] "Agent-Testing Agent: A Meta-Agent for Automated Testing and Evaluation of Conversational AI Agents", arXiv:2508.17393, 2025.
- [5] "Lost in Simulation: LLM-Simulated Users are Unreliable Proxies for Human Users in Agentic Evaluations", arXiv:2601.17087, 2026.
- [6] "Mind the Sim2Real Gap in User Simulation for Agentic Tasks", arXiv:2603.11245, 2026.

- [7] A. Yehudai, L. Eden, A. Li, G. Uziel, Y. Zhao, R. Bar-Haim, A. Cohan y M. Shmueli-Scheuer, "Survey on Evaluation of LLM-based Agents", arXiv:2503.16416, 2025.
- [8] Anthropic, "Building Effective Agents", 2024. [En línea]. Disponible: <https://www.anthropic.com/research/building-effective-agents>
- [9] International AI Safety Report 2026. [En línea]. Disponible: <https://internationalaisafetyreport.org>
- [10] "Towards a Science of AI Agent Reliability", arXiv:2602.16666, 2026.
- [11] Y. Jia y M. Harman, "An Analysis and Survey of the Development of Mutation Testing", IEEE Transactions on Software Engineering, vol. 37, no. 5, pp. 649-678, 2011.
- [12] J. L. Fleiss, "Measuring nominal scale agreement among many raters", Psychological Bulletin, vol. 76, no. 5, pp. 378-382, 1971.
- [13] Y. Benjamini y Y. Hochberg, "Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing", Journal of the Royal Statistical Society, Series B, vol. 57, no. 1, pp. 289-300, 1995.
- [14] Microsoft AI Red Team, "Taxonomy of Failure Modes in Agentic AI Systems", whitepaper v1.0 (2025) y v2.0 (2026). [En línea]. Disponible: <https://www.microsoft.com/en-us/security/blog/2025/04/24/new-whitepaper-outlines-the-taxonomy-of-failure-modes-in-ai-agents/>
- [15] M. Cemri et al., "Why Do Multi-Agent LLM Systems Fail?", arXiv:2503.13657, 2025.
- [16] EvalView, "Open-source regression testing framework for AI agents". [En línea]. Disponible: <https://github.com/hidai25/eval-view>
- [17] Microsoft, "Generate Synthetic and Simulated Data for Evaluation - Microsoft Foundry". [En línea]. Disponible: <https://learn.microsoft.com/en-us/azure/ai-foundry/how-to/develop/simulator-interaction-data>
- [18] LangChain, "LangSmith Evaluation". [En línea]. Disponible: <https://www.langchain.com/langsmith/evaluation>
- [19] OpenTelemetry, "Semantic Conventions for Generative AI Systems". [En línea]. Disponible: <https://opentelemetry.io/docs/specs/semconv/gen-ai/>
- [20] "When Errors Become Narratives: A Longitudinal Taxonomy of Silent Failures in a Production LLM Agent Runtime", arXiv:2606.14589, 2026.
- [21] Braintrust, "Create datasets", documentación oficial. [En línea]. Disponible: <https://www.braintrust.dev/docs/annotate/datasets/create>
- [22] J. Kohl, O. Kruse, Y. Mostafa, A. Luckow, K. Schroer, T. Riedl, R. French, D. Katz, M. P. Luitz, T. Takher, K. E. Friedl y C. Laurent-Winter, "Automated structural testing of LLM-based agents: methods, framework, and case studies", arXiv:2601.18827, 2026.

- [23] A. F. Hayes y K. Krippendorff, "Answering the call for a standard reliability measure for coding data", *Communication Methods and Measures*, vol. 1, no. 1, pp. 77-89, 2007.
- [24] LangChain, "How to simulate multi-turn interactions". [En línea]. Disponible: <https://docs.langchain.com/langsmith/multi-turn-simulation>
- [25] trace2test, proyecto de código abierto, "Turn AI agent execution traces into reproducible regression tests". [En línea]. Disponible: <https://github.com/felmonon/trace2test>
- [26] Sierra Research, " τ^3 -bench", proyecto de código abierto. [En línea]. Disponible: <https://github.com/sierra-research/tau2-bench>

11. Revisión y firma del tutor del proyecto

Yo, Ricardo Flores Moyano, profesor de la carrera de Ingeniería en Ciencias de la Computación, hago constar que he revisado y, por lo tanto, apruebo el documento de planificación del proyecto titulado "AgentProof: inducción automática de escenarios multi-turno desde trazas de interacción y reejecución adaptativa con usuarios simulados para la detección de regresiones en agentes basados en LLM" propuesto por el estudiante Jarod Styph Tierra Medina. Por otra parte, me comprometo a proporcionar al estudiante el soporte necesario y oportuno para el buen desarrollo del proyecto antes mencionado.



Fdo: Ricardo Flores Moyano

Quito, 6 de septiembre de 2026